

IIC2531 2026-1 — Cristóbal Joost

Proyecto Final — ns-exec

El proyecto denominado Unix ns-exec (namespace executor) consiste en el diseño e implementación de un **programa mediador de ejecución** (estilo guardián estudiado en el curso) desarrollado en Python. Busca mejorar la experiencia de usuario en la distribución Kubuntu (versiones 23.10 y 24.04 LTS en adelante) mediante la gestión segura de políticas de confinamiento.

Conocimiento aplicado al caso

Para el desarrollo de esta solución, se investigaron, integraron y aplicaron conceptos fundamentales de la seguridad de sistemas operativos, tales como:

- **Linux Namespaces:** Mecanismos de aislamiento de recursos globales del kernel.
- **Kernel Hardening:** Robustecimiento del kernel enfocado en mitigar vectores de ataque.
- **Sandboxing:** Entornos para ejecutar procesos aislados con privilegios restringidos.
- **Local Privilege Escalation (LPE):** Técnica de ataque en la que un usuario con acceso limitado a un sistema explota una vulnerabilidad o un error de configuración para obtener permisos de administrador.
- **Separación de privilegios:** Segmentación estricta entre el contexto de *root* y el usuario sin privilegios.
- **Linux Security Modules (LSM):** Framework que provee un mecanismo de *hooks* para extensiones de seguridad, permitiendo implementar políticas de Control de Acceso Obligatorio (MAC) e integrarlas directamente en el kernel en tiempo de compilación.

Contexto previo a la implementación

Para contextualizar, es necesario señalar que **AppArmor** es una extensión de seguridad de tipo MAC para el kernel de Linux que implementa una política centrada en tareas, cuyos perfiles son creados y cargados directamente desde el espacio de usuario. A partir de las versiones 23.10 y 24.04 LTS, Canonical introdujo una directiva de *kernel hardening* en Ubuntu destinada a restringir la creación de namespaces de usuarios no privilegiados mediante un parámetro `sysctl` mediado por este módulo. Históricamente, la llamada al sistema `clone(CLONE_NEWUSER)` ha representado una superficie de ataque crítica para la explotación de fallos de LPE al exponer subrutinas internas del kernel a procesos sin credenciales administrativas.

De acuerdo con la documentación oficial publicada en el blog de Ubuntu para la versión 23.10, se reconoce que esta restricción afecta directamente a aplicaciones populares como Firefox y Chrome. Para mitigar este impacto sin forzar el confinamiento total inmediato de las aplicaciones, Canonical introdujo un nuevo modo de perfil en AppArmor denominado `flags=(default_allow)`. Esta directiva permite que un binario específico permanezca operando fuera del confinamiento estándar del módulo de seguridad, pero otorgándole de forma exclusiva la regla `userns`, requerida para el funcionamiento de sus sistemas internos de seguridad.

Sin embargo, la propia documentación advierte explícitamente que esta mitigación es incompleta y fallará si el software se instala a través de empaquetados externos o en una ruta alternativa del sistema de archivos (comportamiento que podría mantenerse en futuras versiones). Este escenario constituyó el origen del problema abordado de forma personal en este proyecto: al utilizar la distribución Kubuntu y contar con binarios instalados en directorios fuera del estándar previsto por AppArmor (como ocurre de forma nativa con la

plataforma Steam o variantes personalizadas de Chromium), el mecanismo de control para crear perfiles por defecto no fue efectivo. Al no existir una coincidencia de ruta exacta en los perfiles cargados, el kernel aplica las restricciones para procesos *unconfined* (`kernel.apparmor_restrict_unprivileged_unconfined=1`), denegando la llamada al sistema y provocando el fallo fatal y el cierre inmediato del software. Por consiguiente, cualquier software ejecutado en un dispositivo con Ubuntu como sistema operativo que, por decisiones del desarrollador, requiera crear namespaces y no cuente con un perfil registrado por el módulo de seguridad, **verá impedida su ejecución**.

Soluciones temporales y sus repercusiones

Las alternativas ante los conflictos de compatibilidad entregadas en la documentación de Ubuntu y por modelos de inteligencia artificial plantean riesgos graves para el estado de seguridad global del sistema:

1. La ejecución de binarios con el flag `--no-sandbox` fuerza a la aplicación a **desactivar sus propios mecanismos internos de aislamiento**, ejecutando código de terceros directamente sobre el contexto del usuario global con protección mínima.
2. La desactivación de la restricción cambiando el parámetro `/proc/sys/kernel/apparmor_restrict_unprivileged_userns` a 0 degrada la postura de seguridad de todo el sistema operativo. Bajo esta configuración, las tareas que no poseen un perfil explícito definido se ejecutan en un estado no confinado (*unconfined state*), el cual es equivalente a estar limitadas únicamente por los permisos DAC estándar de Linux, permitiendo que cualquier proceso malicioso explote la superficie de ataque del kernel mediante, por ejemplo, la creación libre de namespaces.

Objetivo

El objetivo se limita al desarrollo de una herramienta interactiva ejecutable exclusivamente a través de la terminal o mediante invocaciones directas desde gestores de ejecución del entorno de escritorio (como KRunner) tras una configuración específica. Se excluye, por lo tanto, el desarrollo de una GUI compleja.

Para que el programa tenga el efecto explicado en este informe se asume que:

- El sistema base posee AppArmor activo y configurado en modo restrictivo, tal como viene por defecto en la distribución Kubuntu.
- El usuario tiene acceso al dispositivo y conoce la clave de autorización necesaria para ejecutar `sudo` en la terminal, permitiendo las modificaciones del estado del kernel solo si el usuario ingresa las credenciales requeridas (PIN, contraseña o datos biométricos).

Cabe aclarar que, inicialmente, se esperaba desarrollar un programa capaz de validar si el software era seguro y, posteriormente, ejecutarlo con los parámetros requeridos por el sistema operativo. Esta implementación incluía heurística y un sistema externo de validación. Esto no fue posible debido al tiempo destinado al proyecto, pero sí se considera implementable a futuro. Así, el objetivo se enfocó en un código que, antes de ejecutar cualquier binario, solicitara una clave al usuario para luego construir de forma dinámica un perfil para el software, permitiendo su funcionamiento original y resolviendo el conflicto de compatibilidad. Este cambio es necesario para evitar que el usuario deba desactivar la seguridad global o escribir perfiles de AppArmor extensos para cada actualización de software, resolviendo la problemática mediante una lista blanca basada en la integridad de los binarios.

Metodología de desarrollo y declaración de uso de IA

Para la transparencia del proceso técnico y en conformidad con los estándares de integridad académica del curso, se declara el uso de la herramienta de inteligencia artificial generativa Gemini durante el desarrollo de este proyecto. Específicamente, el modelo fue utilizado como soporte para la investigación inicial y análisis de

textos sobre la infraestructura de Ubuntu, la aceleración del diseño conceptual del software, la generación del código base del script `ns-exec.py`, y la redacción de la documentación técnica junto al archivo `README.md`.

Por otra parte, se explicita que las fases críticas de contextualización del problema, el diseño de la arquitectura de seguridad (incluyendo el puente de re-autenticación biométrica), el proceso de pruebas dinámicas y la validación manual de los perfiles de AppArmor fueron ejecutados de forma que la IA no se vió incluida.

Resultados

Se adjunta en el siguiente [enlace](#) el repositorio público que incluye el código de `ns-exec` y un archivo `README.md` con un instructivo, en inglés, pensado para explicar el uso de esta herramienta a conveniencia. Este último contiene al final un [enlace](#) a un video en donde se demuestra la efectividad del programa y el cumplimiento del objetivo en mi dispositivo personal.

Arquitectura y diseño

La arquitectura del programa mediador (`ns-exec.py`) fue pensada y diseñada bajo el modelo de un **Guard** estudiado en el curso, estructurando el flujo en tres fases secuenciales controladas: **Autenticación, Autorización y Mediación**.

1. **Autenticación de entrada:** El programa exige privilegios de superusuario (`root`) mediante `sudo`, delegando la validación de identidad al subsistema PAM del sistema operativo. En resumen, se le solicita la clave o algún dato biométrico para autenticar al usuario.
2. **Resolución de ruta real:** El programa procesa el binario objetivo entregado como parámetro utilizando la llamada `os.path.realpath()` para resolver enlaces simbólicos (*symlinks*) y mitigar ataques de redirección de archivos. Si bien no es el propósito principal, la intención de hacer un programa robusto motivó la investigación e implementación de esta funcionalidad, buscando reducir las brechas que puede llegar a abrir `ns-exec` bajo un uso descuidado.
3. **Validación de integridad criptográfica:** Se calcula el hash **SHA-256** del archivo ejecutable original (siguiendo la ideología TOFU, *Trust on First Use*, al confiar en que el software legítimo es el que se registra por primera vez) en bloques de memoria aislados de 4096 bytes. Este valor se contrasta contra la base de datos protegida en `/etc/usersns-guard/whitelist.json`, la cual cuenta con permisos de control de acceso discrecional (*DAC*) restrictivos (`chmod 600`, propiedad exclusiva de `root`).
4. **Generación dinámica de perfiles (MAC):** Si el hash es válido, el programa construye en memoria una plantilla de AppArmor mínima que declara de forma explícitamente la regla `usersns`, asociada a la ruta real del binario. Este perfil se inyecta directamente al kernel enviando la cadena a través de un canal (*pipe*) hacia el descriptor de entrada estándar (*stdin*) del proceso `apparmor_parser -r`, reduciendo el riesgo de persistencia de archivos residuales en el almacenamiento temporal.
5. **Sanitización de contexto y degradación de privilegios:** Para evitar fallos por *environment pollution*, el programa consulta de forma segura la estructura `/etc/passwd` utilizando el `SUDO_UID` original del proceso. Se corrigen de forma estricta las variables de entorno `HOME`, `USER` y `LOGNAME`. Posteriormente, se invocan de manera atómica las llamadas del sistema `os.setgid()` y `os.setuid()` para privar al proceso de los privilegios de `root`.
6. **Ejecución con `execve()`:** Finalmente, el programa invoca la llamada al sistema `os.execve()`, sustituyendo el espacio de direccionamiento actual por el binario de la aplicación bajo el entorno construido dinámicamente y restringido por AppArmor.

Configuración en el sistema y flujo de uso práctico de ns-exec

La preparación del entorno y el uso diario del programa mediador sigue un flujo secuencial dividido en dos fases principales: la configuración de los archivos mínimos y el ciclo de ejecución de aplicaciones.

En la primera fase, el usuario inicializa la infraestructura de aislamiento creando un directorio restringido en el sistema de archivos global, destinado a almacenar la base de datos de firmas/ hashes. Siguiendo las directrices DAC, este archivo de registro se configura con permisos de lectura y escritura exclusivos para el superusuario, asegurando que procesos no privilegiados no puedan alterar de forma maliciosa las firmas autorizadas. Una vez clonado el repositorio en el almacenamiento local, se le otorgan los permisos de ejecución correspondientes a `ns-exec.py`.

En la segunda fase, es necesario que el usuario ejecute el programa en una terminal, entregando a este la ruta del binario objetivo:

1. **Resolución del binario:** El usuario localiza la ruta absoluta del ejecutable en el disco utilizando herramientas del sistema para evadir cualquier redirección o enlace simbólico intermedio que pudiera comprometer la integridad del análisis.
2. **Registro del binario en *whitelist*:** El usuario invoca el programa mediador a través de la terminal pasando la ruta del binario como argumento. Al ejecutarse, el sistema calcula el hash criptográfico del software. Al detectar que es un binario nuevo, el programa solicita una confirmación interactiva en la terminal; una vez aceptada por el usuario, la firma se almacena de manera persistente en la lista blanca segura.
3. **Ejecución transparente sin restricciones:** Para las ejecuciones subsiguientes, el usuario simplemente llama al binario mediador con la ruta de la aplicación. El programa valida la integridad del archivo instantáneamente, inyecta la política MAC al kernel de Linux para habilitar los namespaces y degrada los privilegios de forma automática, desplegando la aplicación en el espacio de usuario normal con sus defensas nativas intactas.
4. **Eliminar la interacción con terminal para la ejecución:** De forma complementaria, el flujo puede automatizarse modificando los accesos directos del menú del sistema gráfico (como KRunner). Al configurar el acceso directo para invocar al mediador dentro de una terminal interactiva, el usuario puede lanzar sus aplicaciones mediante clics o búsquedas rápidas de la aplicación registrada, requiriendo únicamente la verificación de su huella digital o contraseña a través del subsistema de autenticación para iniciar el entorno seguro.

Limitaciones del proyecto

El diseño actual del programa mediador presenta las siguientes restricciones técnicas:

- El sistema asume la validez absoluta de los datos biométricos y contraseñas gestionados por el módulo PAM local sin verificar posibles suplantaciones físicas previas a la ejecución.
- El programa depende de la integridad de las librerías nativas del intérprete de Python, quedando expuesto a ataques a la cadena de suministro (*supply chain attacks*).
- El programa actúa estrictamente como un validador de integridad criptográfica respecto a un estado previo. Si el usuario registra inicialmente un binario malicioso de forma descuidada, el programa lo considerará legítimo y le otorgará acceso a los recursos del kernel.
- El análisis del comportamiento del hash se limita a una estructura estática, por lo que si se desea confidencialidad en lugar de solo integridad, debería aplicarse una metodología más robusta.

Escalabilidad

El programa mediador presentado opera confiando en el sistema de seguridad del software a ejecutar. AppArmor se utiliza exclusivamente como un atenuador de la restricción global de namespaces. Al incluir reglas permisivas globales para los subsistemas `file`, `network`, `capability`, y `dbus`, el confinamiento granular del proceso queda dependiente de la robustez del *sandbox* interno de la aplicación ejecutada. Esto implica un compromiso parcial del principio de **defensa en profundidad**. Si el aislamiento nativo de la aplicación es vulnerado mediante un exploit, AppArmor no contendrá el compromiso de recursos del sistema operativo. Por lo tanto, para esta entrega se hace el supuesto de que se ejecuta software sin vulnerabilidades explotables.

Como trabajo futuro para el proyecto, se ve posible el desarrollo de una **interfaz de parseo dinámico de políticas**. En lugar de inyectar una plantilla permisiva estática, las iteraciones futuras de `ns-exec.py` deberán ser capaces de:

1. Leer de forma automatizada los perfiles base provistos por los desarrolladores de software legítimo.
2. Fusionar dichas restricciones específicas de archivos y red con la regla de autorización generada por el *Guard* para generar los namespaces.

Esta optimización de arquitectura garantizará un verdadero entorno de mínimo privilegio, donde tanto el kernel del sistema operativo (vía MAC restrictivo) como el proceso hijo coexistan bajo políticas de confinamiento estrictas.

Entorno de trabajo seguro

El desarrollo, testing e implementación de este programa se realizó en un entorno seguro dentro de mi laptop personal. Este entorno consiste en una copia de seguridad del sistema operativo corriendo en una partición aislada dentro del almacenamiento principal. Al operar bajo una arquitectura estrictamente local y aislada, se garantiza que ninguna credencial, token de sesión ni identificador criptográfico del usuario sea expuesto o transmitido fuera del entorno de ejecución nativo del sistema operativo, respetando de forma estricta la privacidad de los datos.

Conclusión

Se puede evidenciar de manera práctica la tensión inherente entre la usabilidad de un sistema operativo orientado a usuario final y las políticas estrictas de seguridad computacional. Las soluciones nativas del sistema operativo tienden a aplicar restricciones absolutas a nivel de kernel para reducir la superficie de ataque, comprometiendo la operatividad de aplicaciones legítimas.

A través de la implementación de este programa mediador, se demostró que es viable arbitrar dinámicamente el acceso a los recursos críticos del kernel del sistema operativo sin comprometer el principio de menor privilegio, aplicando una estrategia de defensa en profundidad fundamentada en controles de acceso obligatorios y criptografía.

Bibliografía

1. <https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces>
2. https://man7.org/linux/man-pages/man7/user_namespaces.7.html
3. <https://attack.mitre.org/tactics/TA0004/>
4. https://chromium.googlesource.com/chromium/src/+/_/0e94f26e8/docs/linux_sandboxing.md
5. <https://docs.kernel.org/admin-guide/LSM/index.html>

6. <https://docs.kernel.org/admin-guide/LSM/apparmor.html>